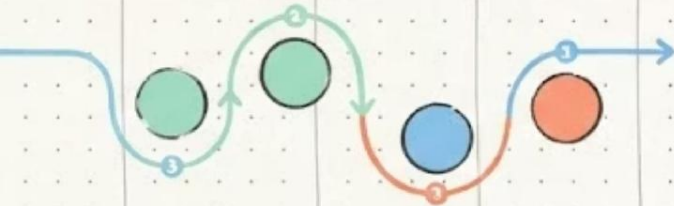


Gestão do Ciclo de Vida da Aplicação

Princípios de Projeto

Professor Romário da Silva Borges



Princípio de Projeto

Responsabilidade Única

Segregação de Interfaces

Inversão de Dependências

Prefira Composição a Herança

Demeter

Aberto/Fechado

Substituição de Liskov

Propriedade de Projeto

Coesão

Coesão

Acoplamento

Acoplamento

Ocultamento de Informação

Extensibilidade

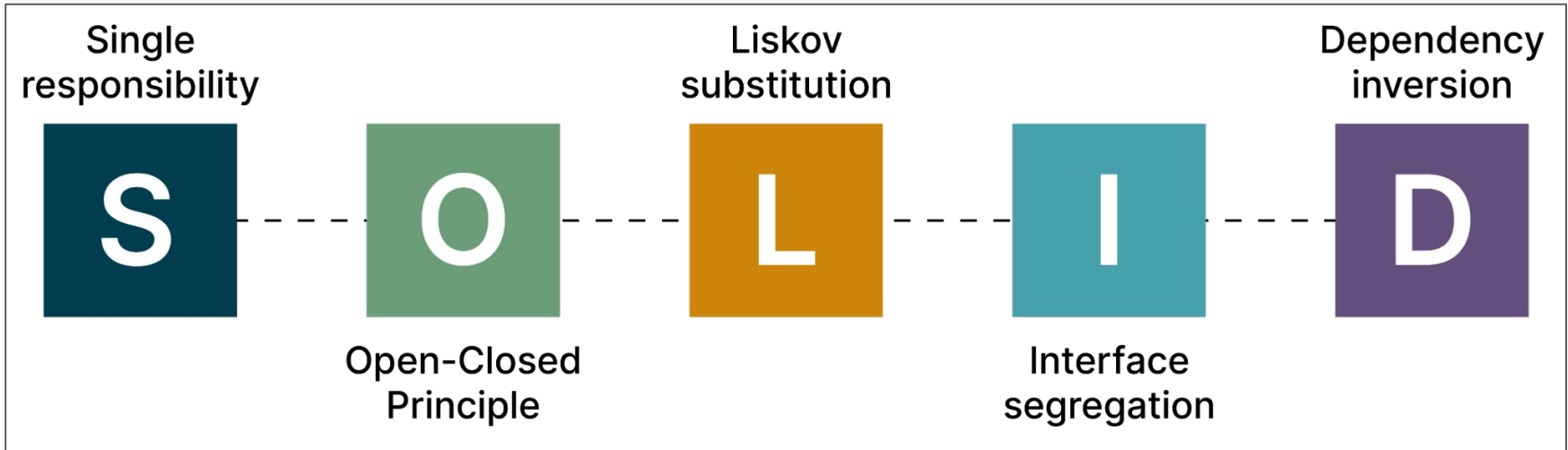
Extensibilidade



Recomendação
mais explícita



Consequência
(o que vamos ganhar
seguindo o princípio)



1 - Princípio da Responsabilidade Única

The background features a series of fluid, overlapping waves in shades of light blue and lavender. These waves create a sense of motion and depth. Interspersed within these waves are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and Java. The code is not the primary focus but adds a technical, digital aesthetic to the overall composition.

```
class Disciplina {
```



```
    void calculaIndiceDesistencia() {  
        indice = "calcula índice de desistência"  
        System.out.println(indice);  
    }
```

```
}
```

Responsabilidade #1: **calcular** índice de desistência

```
class Disciplina {  
  
    void calculaIndiceDesistencia() {  
        indice = "calcula índice de desistência"  
        System.out.println(indice);  
    }  
  
}
```



Responsabilidade #2: **imprimir** índice de desistência

Versão com **separação de responsabilidades**

```
class Console {
```



```
    void imprimeIndiceDesistencia(Disciplina disciplina) {  
        double indice = disciplina.calculaIndiceDesistencia();  
        System.out.println(indice);  
    }
```

```
}
```

```
class Disciplina {
```

```
    double calculaIndiceDesistencia() {  
        double indice = "calcula índice de desistência"  
        return indice;  
    }
```

```
}
```

Classe de apresentação (interface com usuário)
⇒ desenvolvedor frontend

```
class Console {  
  
    void imprimeIndiceDesistencia(Disciplina disciplina) {  
        double indice = disciplina.calculaIndiceDesistencia();  
        System.out.println(indice);  
    }  
  
}  
  
class Disciplina {  
  
    double calculaIndiceDesistencia() {  
        double indice = "calcula índice de desistência"  
        return indice;  
    }  
  
}
```



Classe de negócio (regra de negócio) ou classe de domínio
⇒ desenvolvedor backend
⇒ mais fácil de ser testada (teste de unidade)

Violando SRP em 1 slide



2 - Princípio da Segregação de Interfaces

Segregação de Interfaces

- Interfaces devem ser pequenas, coesas e específicas para cada tipo de cliente
- Caso particular do princípio anterior, mas voltado para interfaces

```
interface Funcionario {  
    double getSalario();  
  
    double getFGTS(); // apenas funcionários CLT  
  
    int getSIAPE(); // apenas funcionários públicos  
  
    ...  
}
```



Versão que atende **segregação de interfaces**



```
interface Funcionario {  
    double getSalario();  
    ...  
}
```

```
interface FuncionarioCLT extends Funcionario {  
    double getFGTS();  
    ...  
}
```

```
interface FuncionarioPublico extends Funcionario {  
    int getSIAPE();  
    ...  
}
```

3 - Princípio da Inversão de Dependências

The background of the slide features a series of flowing, wavy lines in shades of blue and purple, creating a sense of movement and depth. Interspersed within these waves are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and Java, which are slightly blurred to maintain focus on the title.

Inversão de Dependências

- Na verdade, vamos chamar esse princípio de "**Prefira Interfaces a Classes**"
- Pois transmite melhor a sua ideia!

Exemplo sem usar Inversão de Dependências

```
class ControleRemoto {  
    TVSamsung tv;  
    ...  
}  
  
class TVSamsung {  
    ...  
}
```



Qual o possível
problema de design?

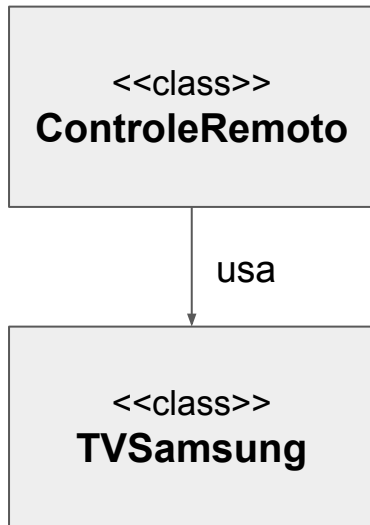
Exemplo usando Inversão de Dependências

```
class ControleRemoto {  
    TVGenerica tv;  
    ...  
}
```

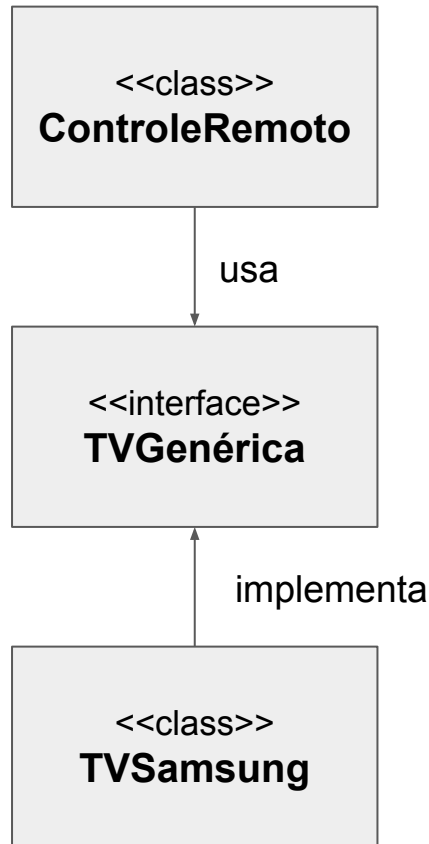
```
interface TVGenerica {  
    ...  
}
```

```
class TVSamsung implements TVGenerica {  
    ...  
}
```

Sem DIP



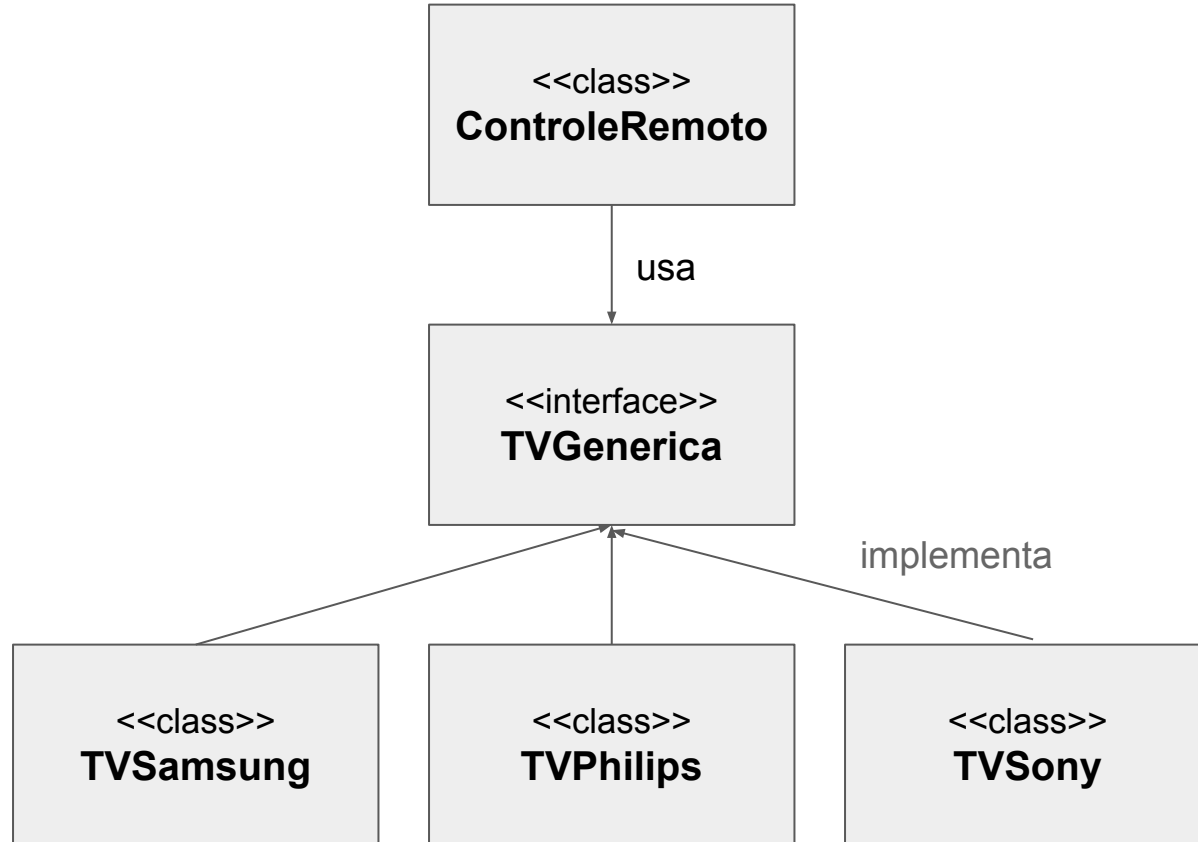
Com DIP



Vantagem: controle remoto genérico, para qualquer TV

Posso mudar de TV, sem alterar o código da classe ControleRemoto

Design após algum tempo



4 - Prefira Composição a Herança

The background features a series of flowing, wavy lines in shades of blue and purple, creating a sense of movement. Faint, semi-transparent snippets of code are visible, particularly on the left and right sides, suggesting a technical or programming context.

Contexto Histórico

- Na década de 80, quando orientação a objetos tornou-se popular, as pessoas começaram a abusar de herança
- Achavam que herança iria ser uma bala de prata, promover reúso em larga escala, etc.

Herança: relação do tipo "é-um"

```
class MotorGasolina extends Motor {  
    ... // herda atributos e métodos de motor  
}
```



Composição: relação do tipo "possui"

```
class Paine1 {  
    ContaGiros cg;  
    ...  
}
```



– referências

```
public class TipoDocumentoIdentificacaoApiModel
{
    – referências
    public int Id { get; set; }
    – referências
    public required string Descricao { get; set; }
}
```

```
public class ListarTipoDocumentoIdentificacaoResponseApiModel : TipoDocumentoIdentificacaoApiModel
{
}
```



ListarTipoDocumentoIdentificacaoRequestApiModel

```
namespace Mersis.WebApi.ApiModels.  
    ApiModel.TipoDocumentoIdentificacao  
{  
    public class ListarTipoDocumentoIdentificacao  
        RequestApiModel  
    {  
        public int? Id { get; set; }  
        public string? Descricao { get; set; }  
        ★ public DadosDocumentoApiModel? DadosDocumento { get; set; }  
    }  
}
```

Composição



DadosDocumentoApiModel

```
public class DadosDocumentoApiModel  
{  
    public string? Numero { get; set; }  
    public string? OrgaoEmissor { get; set; }  
}
```

Prefira Composição a Herança $\Rightarrow$ não force o
uso de herança

5 - Princípio de Demeter

Demeter

- Demeter: nome de um grupo de pesquisa americano
- Evite longas cadeias de chamadas de métodos
- Exemplo:

```
obj.getA().getB().getC().getD().getOqueEuPreciso();
```



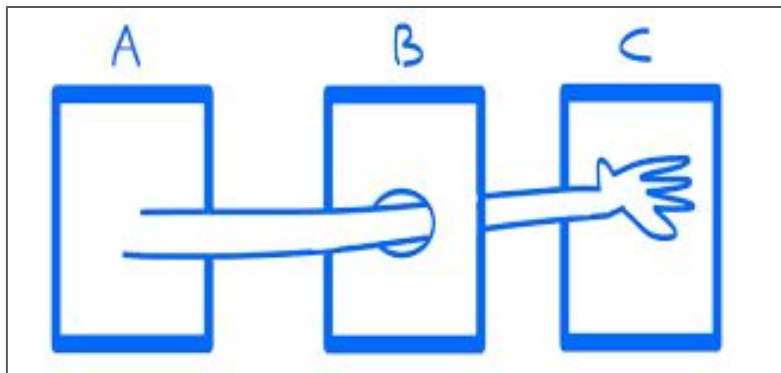
objetos de passagem



Motivo

- Longas cadeias de chamadas quebram encapsulamento
- Não quero passar por A, B, C,... até obter que eu preciso

Violando Demeter



<https://medium.com/@evan.hopkins.us/the-law-of-demeter-and-its-application-to-react-ab1e054f13c5>

```
class PrincipioDemeter {  
  
    T1 attr;  
  
    void f1() {  
        ...  
    }  
  
    void m1(T2 p) { // método que segue Demeter  
        f1();           // caso 1: própria classe  
        p.f2();          // caso 2: parâmetro  
        new T3().f3();    // caso 3: criado pelo método  
        attr.f4();        // caso 4: atributo da classe  
    }  
  
    void m2(T4 p) { // método que viola Demeter  
        p.getX().getY().getZ().doSomething();  
    }  
}
```



Mas cuidado



- Demeter e demais princípios são recomendações
- Não devemos ser radicais e achar que cadeias de chamadas de métodos são totalmente proibidas
- Casos isolados podem existir e terem uma boa justificativa

Exemplo justificável de cadeias de métodos

```
const numbers = [1, 2, 3, 4, 5, 6];  
const result = numbers  
  .map(num => num * 2)  
  .filter(num => num % 3 === 0)  
  .reduce((acc, num) => acc + num, 0);  
console.log(result); // Output: 18
```



Métodos da linguagem
ou de sua API

6 - Princípio Aberto/Fechado

The background features a light blue gradient with several wavy, horizontal bands of color in shades of purple, blue, and teal. These bands are overlaid with faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and C++, creating a technical and modern aesthetic.

Princípio Aberto/Fechado

- Proposto por Bertrand Meyer
- Ideia: uma classe deve estar **fechada** para modificações, mas **aberta** para extensões



1988

Explicando melhor

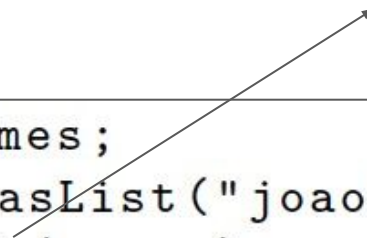
- Suponha que você vai implementar uma classe
- Usuários ou clientes vão querer usar a classe (óbvio!)
- Mas vão querer também customizar, parametrizar, configurar, flexibilizar e estender a classe!
- Você deve se antecipar e tornar possível tais extensões
- Mas sem que os clientes tenham que editar o código da classe

Como tornar uma classe **aberta** a extensões, mas mantendo o seu código **fechado** para modificações?

- Parâmetros
- Funções de mais alta ordem
- Padrões de projeto
- Herança
- etc

Exemplo

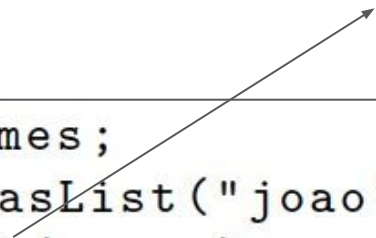
Ordena a lista passada com parâmetro



```
List<String> nomes;  
nomes = Arrays.asList("joao", "maria", "alexandre", "ze");  
Collections.sort(nomes);
```

Exemplo

Ordena uma lista passada com parâmetro



```
List<String> nomes;  
nomes = Arrays.asList("joao", "maria", "alexandre", "ze");  
Collections.sort(nomes);
```

```
System.out.println(nomes);  
// resultado: ["alexandre","joao","maria","ze"]
```

Mas agora eu quero ordenar as strings da lista pelo seu tamanho, isto é, pelo número de chars

Será que o método sort está aberto (preparado) para permitir essa extensão?

Mas, mantendo o seu código fechado, isto é, sem ter que mexer no seu código

Solução

lista que será
ordenada

Função usada por sort para comparar s1 e s2:

- $\text{result} < 0 \Rightarrow$ ordem é s1, s2
- $\text{result} = 0 \Rightarrow$ tanto faz a ordem
- $\text{result} > 0 \Rightarrow$ ordem é s2, s1

```
Collections.sort(nomes, (s1, s2) -> s1.length() - s2.length());
```

```
System.out.println(nomes);
```

```
// result: ["ze", "joao", "maria", "alexandre"]
```



Não existe almoço grátis: cliente (código que chama "sort") teve que implementar e passar como parâmetro uma função que define o critério a ser usado na ordenação

Resumindo: ao implementar uma classe, pense em pontos de extensão!

7 - Princípio de Substituição de Liskov

The background of the slide features a light blue gradient with several wavy, horizontal bands of color in shades of purple, blue, and teal. These bands are overlaid with faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and Java. The code is mostly illegible due to the low opacity and the wavy motion blur effect.

Princípio de Substituição de Liskov

- Nome é uma referência à Profa. Barbara Liskov
- Princípio define boas práticas para uso de herança
- Especificamente, boas práticas para redefinição de métodos em subclasses



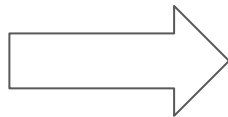
Primeiro: vamos entender o termo "substituição"

```
void f(A a) {  
    ...  
    a.g(int n);  
    ...  
}
```

```
void f(A a) {  
    ...  
    a.g(int n);  
    ...  
}
```

```
f(new B1()); // f pode receber objetos da subclasse B1  
...  
f(new B2()); // e de qualquer outra subclasse de A, como B2  
...  
f(new B3()); // e B3
```

```
void f(A a) {  
    ...  
    a.g(int n);  
    ...  
}
```



- Tipo A pode ser substituído por B1, B2, B3,...
- Desde que eles sejam subclasses de A

Princípio de Substituição de Liskov

- Substituições de A por B podem acontecer desde que B ofereça os mesmos serviços de A
- Ou seja: para um código feito para usar A, a substituição de A por B é “imperceptível”

Exemplo que segue Substituição de Liskov

```
class ControleRemoto {  
    // alcance de 10 m  
}  
  
class ControleRemotoPremium extends ControleRemoto {  
    // alcance de 20 m  
}
```



Exemplo que **não** segue Substituição de Liskov

```
class ControleRemoto {  
    // alcance de 10 m  
}  
  
class ControleRemotoXYZ extends ControleRemoto {  
    // alcance de 5 m  
}
```



Atividades

The background features a series of flowing, wavy lines in shades of blue and purple, creating a sense of movement and depth. Faint, semi-transparent snippets of code are visible, particularly on the left and right sides, suggesting a technical or programming theme. The overall aesthetic is clean and modern.

1. Qual princípio de projeto é violado por uma chamada como a mostrada abaixo? Qual mudança de projeto você faria na classe Biblioteca (tipo de bib) para remover essa violação?

```
bib.getAcervo().getAreaConhecimento("ES").getLivros().find("ESM").getNumExemplares();
```

Princípio violado: Lei de Deméter (Law of Demeter / "Don't talk to strangers");

viola esse princípio porque o código cliente precisa conhecer toda a cadeia de objetos internos de Biblioteca — *Acervo*, *AreaConhecimento*, *a coleção de Livros* e o *próprio Livro* — para chegar até o dado que realmente quer (*getNumExemplares()*).

Isso **quebra o encapsulamento**: o cliente passa a depender da estrutura interna de Biblioteca e de todas as classes intermediárias, não apenas da interface pública dela.

Solução: criar em Biblioteca um método que encapsule a navegação interna e devolva diretamente o resultado desejado:

```
public class Biblioteca {  
    private Acervo acervo;  
  
    public int getNumExemplares(String codigoArea, String codigoLivro) {  
        return acervo.getAreaConhecimento(codigoArea).getLivros().find(codigoLivro)  
            .getNumExemplares();  
    }  
}
```

O código cliente passa a fazer apenas:

```
bib.getNumExemplares("ES", "ESM");
```

2. Suponha que você está implementando um sistema S e precisa usar um serviço de processamento de pagamentos X para, por exemplo, aceitar pagamentos via cartões de crédito.

No entanto, você não quer acoplar S a X, pois futuramente você pode optar por um outro serviço Y.

- (a) Explique como você faria para desacoplar S de X.
- (b) Qual princípio de projeto você usou para garantir esse desacoplamento?

3 - Imagine o seguinte cenário: você está desenvolvendo um pequeno sistema interno para uma startup — um script que, uma vez por mês, gera um relatório de vendas e envia por e-mail usando o Gmail (via SMTP). É a única forma de envio de e-mail que a empresa usa, não há planos de trocar de provedor, o sistema é usado por 3 pessoas, e o requisito é simplesmente "enviar o relatório todo mês".

Faria sentido usar DIP nesse caso?

Em Engenharia de Software, às vezes usamos soluções mais complexas em contextos nos quais isso não é necessário. Esse problema é chamado de overengineering ou otimização prematura.

Professor Marco Tulio Valente

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: <https://engsoftmoderna.info>.

BEZERRA, E.

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

PAULA, Filho W. P.

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

HORSTMANN, C.

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

SOMMERVILLE, I.

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

PRESSMAN, R.

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000

